CCS231 - Parallel and Distributed Computing

System Design Document

Distributed Publish-Subscribe Messaging Middleware

Abstract

This document details the architecture and implementation of a custom Python-based Publish-Subscribe messaging system. Designed to handle distributed event routing, the system utilizes a centralized broker architecture communicating over raw TCP sockets. To guarantee fault tolerance, the middleware implements at-least-once delivery semantics supported by a normalized, durable SQLite queue. Advanced features include a 4-byte length-prefix framing protocol to manage TCP stream fragmentation (IPC), hierarchical topic-based routing, and robust state-recovery mechanisms for disconnected clients. The system is demonstrated via a real-time stock market simulation featuring edge-triggered alerting and dynamic mean-reversion.

Submitted by:

Aquino, Dallas A.
Buñag, Frederick Jibril L.
Carbonell, Ethan Jed V.
Corpes, Vincent L. Jr.

Instructor: Mr. Cyreneo S. Dofitas

Course: CCS231 - Parallel and Distributed Computing

Date: April 2026

Contents

1 Introduction

In modern distributed systems, decoupling the producers of data from the consumers is critical for scalability and fault tolerance. This project implements a robust Publish-Subscribe (Pub/Sub) middleware from scratch. The primary objective of this system is to route messages accurately while guaranteeing that no data is lost during network failures or client crashes. To demonstrate the system's capabilities, a real-world scenario was developed: a Live Stock Market Feed. In this simulation, publishers generate volatile market data (including calculated market shocks and mean-reversion algorithms), while multiple subscribers—including a CLI monitoring tool and a real-time Web Dashboard—consume, filter, and display the data using edge-triggered threshold logic.

2 System Architecture

2.1 Centralized Broker Pattern

The system employs a **Centralized Broker Pattern**. Rather than clients establishing peer-to-peer connections, all publishers and subscribers maintain a single, persistent TCP connection to the central Broker. This topology ensures strict decoupling; publishers are completely unaware of who is receiving their data, and subscribers are unaware of the data's origin.

2.2 Component Breakdown

The Broker is highly modularized, utilizing a Thread Pool pattern (up to 50 concurrent workers) to prevent thread exhaustion. The core logic is divided into distinct managers, as illustrated in Figure 1:

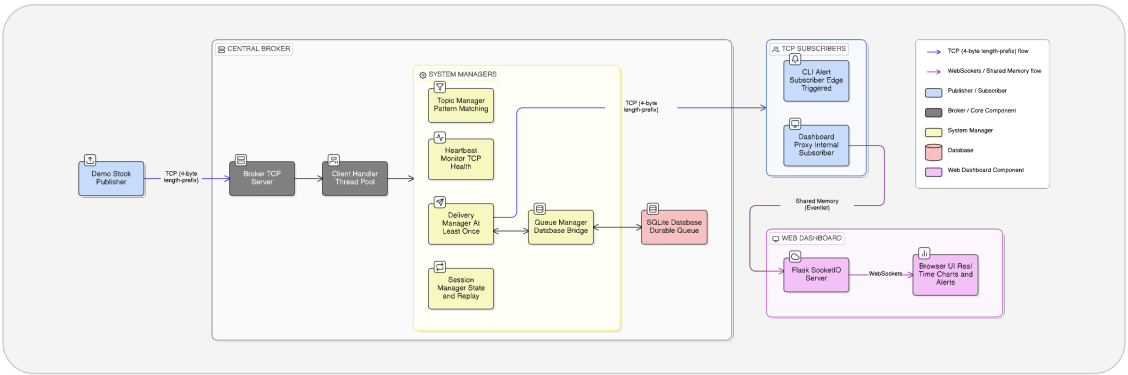

Figure 1: High-Level System Architecture showing the Broker's internal managers, thread pool, durable queue, and client interfaces.

As shown in the diagram, incoming TCP connections are immediately handed off to the **Client Handler Thread Pool**. From there, message payloads and states are delegated to five primary system managers:

- **Topic Manager:** Handles the routing logic, matching incoming published topics (e.g., `STOCK.AAPL`) against subscriber patterns (including wildcards like `STOCK.*`).
- **Delivery Manager:** Enforces the at-least-once delivery guarantee by persisting messages to disk before transmission and executing background retry loops for unacknowledged messages.
- **Queue Manager:** Acts as the bridge between the domain logic and the SQLite persistence layer.
- **Session Manager:** Handles state resumption. Upon client reconnection, it queries the durable queue and replays missed messages.
- **Heartbeat Monitor:** Actively tracks client health, cleanly severing dead TCP sockets if a client misses two consecutive 10-second heartbeats.

3 Inter-Process Communication (IPC)

3.1 Transport Layer

The foundation of the middleware relies on raw TCP (Transmission Control Protocol) sockets for all Inter-Process Communication. TCP was chosen over UDP to leverage its native reliability and packet-ordering guarantees, which are essential for financial market data.

3.2 Custom Framing Protocol

A common pitfall in distributed systems programming is treating TCP as a packet-based protocol rather than a continuous byte stream. If a publisher sends two JSON payloads in rapid succession, the OS networking stack may merge them into a single transmission (TCP stream fragmentation/-concatenation), causing standard JSON parsers to crash. To resolve this, we engineered a custom **4-byte length-prefix framing protocol**. Before any JSON string is transmitted, it is encoded into bytes (UTF-8). The exact length of this byte array is calculated and packed into a 4-byte big-endian integer (`!I`) header.

Header (4 Bytes, Big-Endian)	**Payload (N Bytes, UTF-8)**
e.g., `0x00 0x00 0x00 0x4B` (Length: 75)	`{"type": "PUBLISH", "topic": "..."}`

Figure 2: Structure of the custom TCP framing packet.

On the receiving end, the `recv_message` function enters a strictly controlled `while` loop:

1. It reads exactly 4 bytes to unpack the payload length.
2. It loops reading the socket until exactly N bytes of the payload have been received.

This ensures 100% safe deserialization of JSON payloads across distributed nodes, completely isolating the application from underlying network segmentation.

4 Routing Mechanisms

A critical design decision in any Publish-Subscribe system is determining how messages are routed to interested clients.

4.1 Design Decision: Topic-Based vs. Content-Based Filtering

We explicitly chose to implement **Topic-Based Filtering** over **Content-Based Filtering**.

- **Content-Based Filtering** requires the broker to deserialize and inspect the actual data payload of every incoming message to evaluate subscriber rules (e.g., `SUBSCRIBE WHERE payload.price > 200`). While highly flexible for clients, this forces the broker to perform computationally expensive evaluations on every transmission, severely limiting overall system throughput.
- **Topic-Based Filtering** relies on hierarchical string labels attached to the message exterior (e.g., `STOCK.AAPL`). The broker acts purely as a fast router. It does not need to parse the JSON payload; it simply checks the string label against a hash map of subscribers.

By utilizing Topic-Based filtering, we maintain high throughput and ensure the Broker remains entirely agnostic to the schema of the payloads it routes. To provide flexibility, the `TopicManager` fully supports wildcard routing, allowing clients to subscribe to exact topics (`STOCK.TSLA`) or multi-level domains (`STOCK.*`).

5 Fault Tolerance and Delivery Guarantees

In a distributed environment, network partitions and client crashes are inevitable. This system guarantees **At-Least-Once Delivery**. If a subscriber goes offline, the system ensures no messages destined for that subscriber are lost.

5.1 Durable Queues and Database Normalization

Message persistence is backed by an embedded SQLite database. A naive approach to durable queues involves saving a copy of the JSON payload for every subscriber that needs it. If 10 subscribers are listening to `STOCK.AAPL`, saving the heavy payload 10 times results in severe disk I/O bottlenecks. To optimize performance, we designed a **Normalized Database Schema**, depicted in Figure 3:

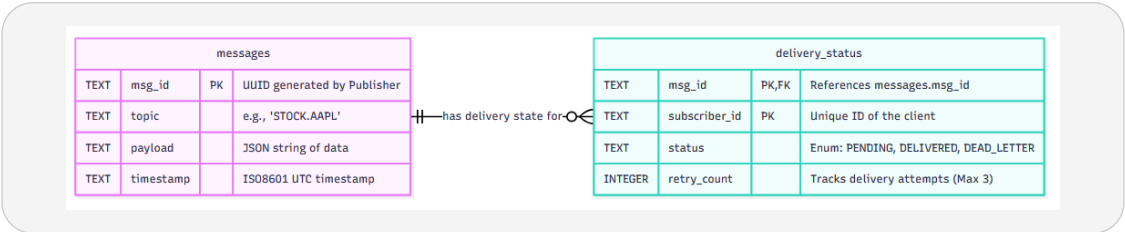

Figure 3: Entity-Relationship diagram of the Normalized SQLite Durable Queue.

By utilizing a one-to-many relationship, the database strictly separates the heavy payload from the client state:

1. **`messages` Table:** Stores the heavy, immutable JSON payload exactly once, indexed by a UUID (`msg_id`).
2. **`delivery_status` Table:** A lightweight junction table that maps the `msg_id` to a specific `subscriber_id`. This table tracks the state (`PENDING`, `DELIVERED`, or `DEAD_LETTER`) and the `retry_count`.

5.2 Delivery and Recovery Flow

The actual sequence of events required to fulfill the At-Least-Once delivery guarantee is highly dependent on database state mutations and client acknowledgments (ACKs). Figure 4 illustrates both a standard delivery flow and a fault-tolerance recovery flow.

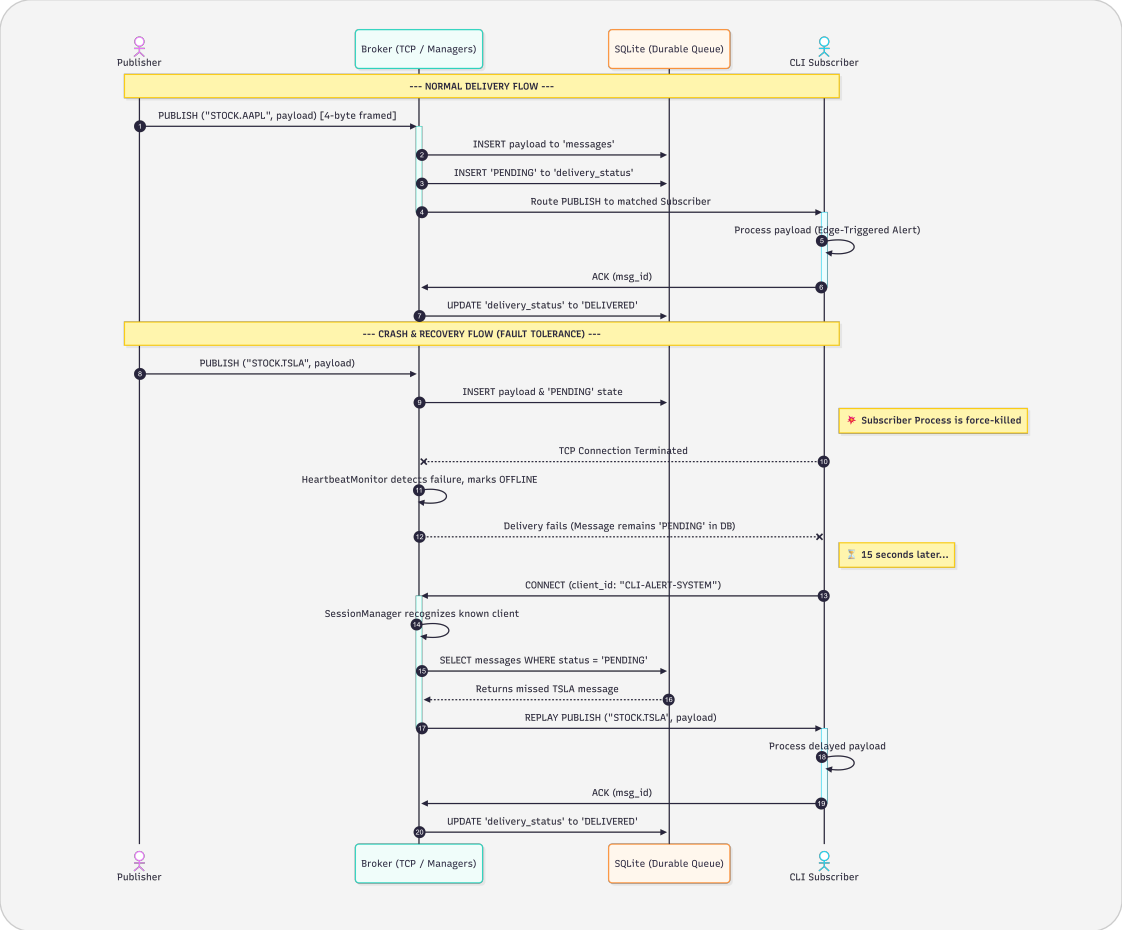

Figure 4: Sequence Diagram showing the At-Least-Once Delivery protocol and state-recovery mechanisms following a client crash.

As visualized above, when the broker receives a `PUBLISH` message, it executes the following sequence:

1. The payload is inserted into the `messages` table.
2. For every matching subscriber, a `PENDING` state is inserted into `delivery_status`.
3. The broker attempts to transmit the message over the TCP socket.
4. Upon successful processing, the client replies with an `ACK` message containing the `msg_id`.
5. The broker receives the `ACK` and updates the database state to `DELIVERED`.

If a client crashes and misses its heartbeats (as seen in the bottom half of Figure 4), its socket is closed. The messages remain safely marked as `PENDING` on disk. Upon reconnection, the `SessionManager` identifies the client, queries the database for all `PENDING` messages, and immediately replays the missed history. If a message exhausts its `MAX_RETRIES` limit, it is removed from the active delivery queue and marked as `DEAD_LETTER` for administrative review, preventing “poison pill" messages from clogging the system.

6 Real-World Scenario: Live Market Alerts

To validate the middleware, a realistic stock market simulation was developed comprising a Publisher, a CLI application, and a Web Dashboard.

6.1 Market Simulation (Mean Reversion)

The `stock_publisher.py` module generates data using a random-walk algorithm modified with a “Mean Reversion" (gravity) mathematical model. When random market shocks cause extreme price spikes, gravity naturally pulls the stock back to its baseline. This prevents prices from drifting to infinity and creates a continuous stream of realistic, volatile data for the system to process.

6.2 Edge-Triggered Alerting Logic

Both the CLI Subscriber and the internal proxy for the Web Dashboard utilize **Edge-Triggered Logic** for their threshold alerts. Unlike level-triggered systems that continuously spam alerts as long as a condition is met (e.g., price $>$ \$200), our Edge-Triggered implementation tracks state changes. It fires a critical alert only at the exact moment the threshold is breached, remains silent while the price stays high, and fires a green “Recovery" alert the exact moment the price drops back into the safe zone. This logic creates a highly professional user experience and visually proves that duplicate queue replays are handled intelligently by the client application.

6.3 Web Dashboard Interface

To provide comprehensive visibility into the system’s distributed state, a real-time Web Dashboard was implemented using Flask and WebSockets. Figure 5 showcases the interface during an active market simulation.

Figure 5: Sequence Diagram showing the At-Least-Once Delivery protocol and state-recovery mechanisms following a client crash.

The dashboard acts as a specialized subscriber, directly reflecting the health and throughput of the middleware. The **Live Price Charts** and **Message Feed** update instantaneously as payloads arrive via the custom TCP protocol. The **Broker Status** panel subscribes to a dedicated internal system topic (`$SYS.BROKER.STATS`), allowing administrators to monitor active connections and verify the Durable Queue's behavior in real-time by observing the `Pending ACKs` metric. Finally, the **Alert Log** visually demonstrates the edge-triggered threshold logic, flashing with corresponding severity colors during market shocks and stabilization events.

7 Conclusion

This project successfully bridges fundamental distributed systems theory with practical software engineering. By combining IPC via custom TCP framing, Topic-Based filtering, and rigorous At-Least-Once delivery guarantees backed by normalized durable queues, the resulting middleware is both highly performant and exceptionally resilient to node failures.